

Pattern Matching and Machine Learning for Audio Signal Processing

Prof. Dr. Frank Kurth

Prof. Dr. Michael Clausen

INSTITUTE OF COMPUTER SCIENCE, UNIVERSITY OF BONN

Chapter 6: Generative classification via HMMs

Chapter 6: Generative classification via HMMs	2
Preliminary remarks	2
Discrete probability spaces	3
Bayes rule and notational conventions	4
Generative vs. discriminative classifiers	5
Audio-based chord recognition	6
Illustration of the major and minor triads	7
Illustration of the chord recognition task	8
A simple template-based chord recognition strategy	9
A basic approach to feature extraction	10
A basic approach to template matching	11
Binary chord templates & cosine similarity	12
Template-based chord recognition: an example	13
Techniques for improving recognition	14
Improved templates with harmonics	15
Improved templates from examples	16
Feature enhancement strategies	17
Enhanced chroma features	18
Evaluation measures	19
Training and validation	20
Effects of temporal smoothing	21
Context-aware classification schemes	22
Markov chains: a motivating example	23
Markov chains: the model	24
Illustration of a Markov chain	25
Markov chains: an example	26
The need to extend the Markov model	27
Hidden Markov models	28
Illustration of an HMM	29
HMM-based generation of observations	30

HMM: the probabilistic setting	31
HMM: evaluation and model specification	32
HMM: the evaluation problem	33
Efficient evaluation: the forward procedure	34
Proof of the induction formula	35
Forward procedure: trellis diagram.	36
HMM: the uncovering problem	37
Viterbi algorithm: preliminaries (1)	38
Viterbi algorithm: preliminaries (2)	39
Viterbi algorithm: preliminaries (3)	40
The Viterbi algorithm	41
Trellis diagram for computing P^*	42
Trellis diagram for computing S^*	43
Illustration of the Viterbi algorithm	44
The estimation problem: preliminaries	45
The Baum-Welch algorithm (1)	46
The Baum-Welch algorithm (2)	47
Maximum-likelihood estimation of HMM parameters	48
HMM-based chord recognition	49
Automated specification of Transition Probabilities	50
Example: estimated transition probability matrix	51
Transposition-invariant transition probabilities	52
Specification of observation symbols	53
Continuous HMMs	54
References	55

Preliminary remarks

- **Classification problems are omnipresent.** Here are some examples:
 - ◆ **text categorization** (e.g., for spam filtering)
 - ◆ **fraud detection** (e.g., to prevent money laundering)
 - ◆ **optical character recognition** (e.g., for sorting letters on the basis of machine-read postcodes)
 - ◆ **machine vision** (e.g., for face detection)
 - ◆ **natural language processing** (e.g., for spoken language understanding)
 - ◆ **environmental sound classification** (e.g., for indicating species diversity, abundance and overall environmental health)
 - ◆ ...
- **Classification** is the task to assign a **class** $y \in \mathcal{Y}$ to an **observation** $x \in \mathcal{X}$.
- We will focus on **probabilistic classifiers** that specify a probability distribution to select the most likely class y for a given observation x . In the **supervised learning** scenario, this specification is based on **training examples** of the form (x_i, y_i) where x_i is the **feature vector** of the i -th example and y_i is its **label** (i.e., class).

Lecture PM2L4 Audio

Chapter 6: Generative classification via HMMs – 2 / 55

Discrete probability spaces

We start by recalling basic notions and facts from probability theory.

Definition 1.

- A **discrete probability space** is a pair (Ω, P) consisting of a finite or countably infinite set Ω , the **sample space**, and a **probability measure** P , which is a function $P : \Omega \rightarrow [0, 1]$ satisfying $\sum_{\omega \in \Omega} P(\omega) = 1$. Each subset E of Ω is called an **event**, and $P(E) := \sum_{e \in E} p(e)$ measures the probability of the event E .
- Let (Ω, P) denote a discrete probability space and E an event with $P(E) > 0$. Then the **conditional probability of the event F given E** is defined as the quotient

$$P(F|E) := \frac{P(F \cap E)}{P(E)}.$$

Theorem 2. Let (Ω, P) denote a discrete probability space and E an event with $P(E) > 0$. Then the map $\Omega \ni \omega \mapsto P(\omega|E) := P(\{\omega\}|E)$ defines a new probability measure on Ω .

Proof. $\sum_{\omega \in \Omega} P(\omega|E) = \sum_{\omega \in \Omega} \frac{P(\{\omega\} \cap E)}{P(E)} = \sum_{e \in E} \frac{P(e)}{P(E)} = \frac{P(E)}{P(E)} = 1.$ □

Lecture PM2L4 Audio

Chapter 6: Generative classification via HMMs – 3 / 55

Bayes rule and notational conventions

Theorem 3. (Bayes rule) Let (Ω, P) denote a discrete probability space and let A and B be events with $P(A) > 0$ and $P(B) > 0$. Then

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}.$$

Proof. This follows from the definition of conditional probability. $\square$

Freshmen in machine learning are often confronted with a stochastic dialect that is hard to understand. The following comments serve as a translation device.

- Let $\mathcal{X} = \mathcal{X}_1 \times \dots \times \mathcal{X}_m$ denote the set of observations (e.g., feature vectors) and $\mathcal{Y}$ the finite set of class labels. Then $\Omega = \mathcal{X} \times \mathcal{Y}$ is the sample space.
- It is common practice to use for $x \in \mathcal{X}$, $y \in \mathcal{Y}$, and $A_i \subset \mathcal{X}_i$ the following identifications:

$$x \equiv \{x\} \times \mathcal{Y} \quad y \equiv \mathcal{X} \times \{y\}$$

$$A_i \equiv \mathcal{X}_1 \times \dots \times \mathcal{X}_{i-1} \times A_i \times \mathcal{X}_{i+1} \times \dots \times \mathcal{X}_m \times \mathcal{Y}.$$

Generative vs. discriminative classifiers

- Let $\mathcal{X}$ denote the set of observations (e.g., feature vectors) and $\mathcal{Y}$ the finite set of class labels. Then $\Omega = \mathcal{X} \times \mathcal{Y}$ is the sample space.
- **Generative classifiers** learn via training examples a model of the **joint probability**, $P(x, y)$, of the inputs x and the label y and make their predictions using Bayes rules to calculate $P(y|x) := P(\mathcal{X} \times \{y\} | \{x\} \times \mathcal{Y})$ by

$$P(y|x) = \frac{P(y) \cdot P(x|y)}{P(x)}$$

and then picking the most likely label y . In Bayesian probability terminology, $P(y|x)$ is called the **posterior**, $P(y)$ the **prior**, $P(x|y)$ the **likelihood**, and $P(x)$ the **evidence**.

- **Discriminative classifiers** model the posterior $P(y|x)$ directly, or learn a direct map from the inputs x to the class labels.
- In this chapter we will discuss **Hidden Markov Models** (HMMs), which are generative classifiers.
- In the next chapter we will discuss **Support Vector Machines** (SVMs), which are discriminative classifiers.

Audio-based chord recognition

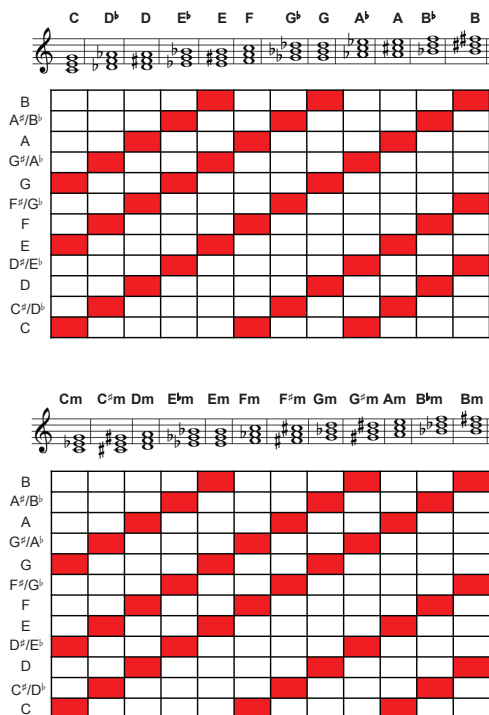
- The **audio-based chord recognition problem** will serve as a friendly guide to the **Hidden Markov Model**.
- We assume that the piece of music is given in the form of an audio recording.
- The **chord recognition task** consists in splitting up the recording into segments and assigning a chord label to each segment. The segmentation specifies the start and end time of a chord, and the chord label specifies which chord is played during this time period.
- To keep things as simple as possible, we consider only a small number of the most important chords as occurring in Western music. More precisely, we will restrict ourselves to all 12 **major triads** and all 12 **minor triads**:

C C[#] D D[#] E F F[#] G G[#] A A[#] B
Cm C[#]m Dm D[#]m Em Fm F[#]m Gm G[#]m Am A[#]m Bm

Here we have identified **C[#]** and **D^b**, etc. (→ **enharmonic equivalence**).
 For more details see the next slide.

Illustration of the major and minor triads

Overview of all major triads (top) and all minor triads (bottom) up to enharmonic equivalence and octave shifts.



- **major triad:** (e.g., C – E – G)

root $\xrightarrow[3 \text{ semitones}]{4 \text{ semitones}}$ **second note**
 $\xrightarrow[minor \text{ third}]{major \text{ third}}$ **third note.**

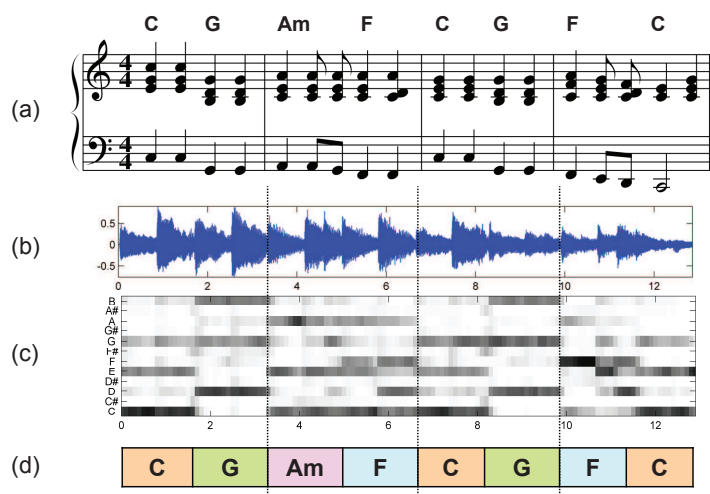
- **minor triad:** (e.g., C – E^b – G)

root $\xrightarrow[4 \text{ semitones}]{3 \text{ semitones}}$ **second note**
 $\xrightarrow[major \text{ third}]{minor \text{ third}}$ **third note.**

- One possible score notation is given for each chord, as well as a chroma pattern encoding the respective three-element subset (indicated by the red cells).

Illustration of the chord recognition task

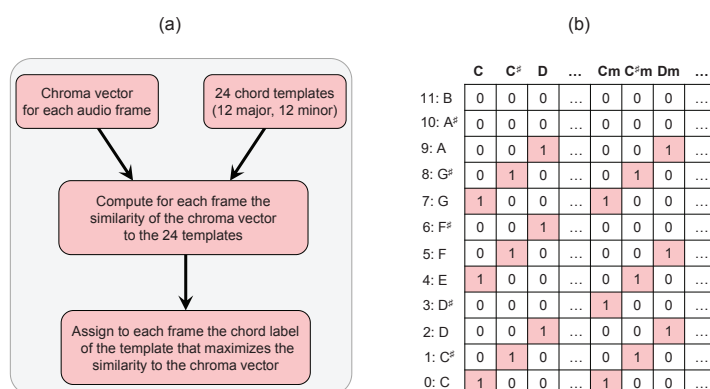
- We illustrate the chord recognition task by the first four measures of the **Beatles song “Let It Be”**.



(a) Score of the first four measures. (b) Waveform of an audio recording of these measures. (c) Chroma representation. (d) Chord recognition result.

A simple template-based chord recognition strategy

- We start with a simple template-based chord recognition strategy.
- Then we will indicate improvements of this procedure.
- These improvements will culminate in an **HMM-based chord recognizer**.



- (a) Overview of a simple template-based chord recognition procedure.
- (b) Binary chord templates for major and minor triads.

On the next slides we will work out the details of this procedure.

A basic approach to feature extraction

- Given an audio recording of a piece of music, the goal of our chord recognition task is to find out which chords are played at which time.
- The first step is to transform the recording into a sequence $X = (x_1, x_2, \dots, x_N)$ of feature vectors $x_n \in \mathcal{F}$, $n \in [1 : N]$, where $\mathcal{F}$ denotes a suitable feature space.
- As chroma-based features capture a signal's short-time tonal content, which is closely correlated to the harmonic progression of the underlying piece, we use the **chromagram feature** (Chapter 4, Slide 11) in our basic approach.
- Recall that each chromagram feature is a vector

$$x = (x(0), x(1), \dots, x(11))^T \in \mathcal{F} \quad \text{with } \mathcal{F} = \mathbb{R}^{12}.$$

- In our examples, we use a **window size** that corresponds to 200 ms and a **hop size** of half the window length.
- Thus our feature sequence $X = (x_1, x_2, \dots, x_N)$ has a **feature rate** of 10 Hz.

A basic approach to template matching

- In the end, each feature vector x_n is to be mapped to a chord label $\lambda_n \in \Lambda$. Here, we restrict ourselves to the set of the twelve major and minor triads:

$$\Lambda = \{\mathbf{C}, \mathbf{C}^\sharp, \mathbf{D}, \dots, \mathbf{B}, \mathbf{Cm}, \mathbf{C}^\sharp\mathbf{m}, \mathbf{Dm}, \dots, \mathbf{Bm}\}.$$

In this case, each frame $n \in [1 : N]$ is assigned to a major or a minor triad specified by λ_n .

- The **general matching idea** is to precompute a set $\mathcal{T} \subset \mathcal{F} = \mathbb{R}^{12}$ of **templates** denoted by $t_\lambda \in \mathcal{T}$, $\lambda \in \Lambda$. Intuitively, each template can be thought of as a prototypical chroma vector that represents a specific musical chord. Furthermore, we fix a **similarity measure** $s : \mathcal{F} \times \mathcal{F} \rightarrow \mathbb{R}$ that allows for comparing different chroma vectors.
- Then, the template-based procedure consists in assigning the chord label that maximizes the similarity between the corresponding template and the given feature vector x_n :

$$\lambda_n := \operatorname{argmax}_{\lambda \in \Lambda} s(t_\lambda, x_n).$$

Binary chord templates & cosine similarity

- In our basic approach, we use the **binary chord templates** for major and minor triads. These are the 24 columns of the matrix on Slide 9 (read bottom-up!). For example,

$$\begin{aligned} t_C &= (1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0)^T \in \mathbb{R}^{12} \\ t_{Dm} &= (0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0)^T \in \mathbb{R}^{12}. \end{aligned}$$

- There are many ways for comparing chroma features and chord templates. Because of its simplicity, we use the **cosine similarity measure**, which is the inner product of normalized vectors

$$s(x, y) := \frac{\langle x | y \rangle}{\|x\| \cdot \|y\|} = \cos(x, y).$$

If $\|x\| \cdot \|y\| = 0$, we set $s(x, y) := 0$.

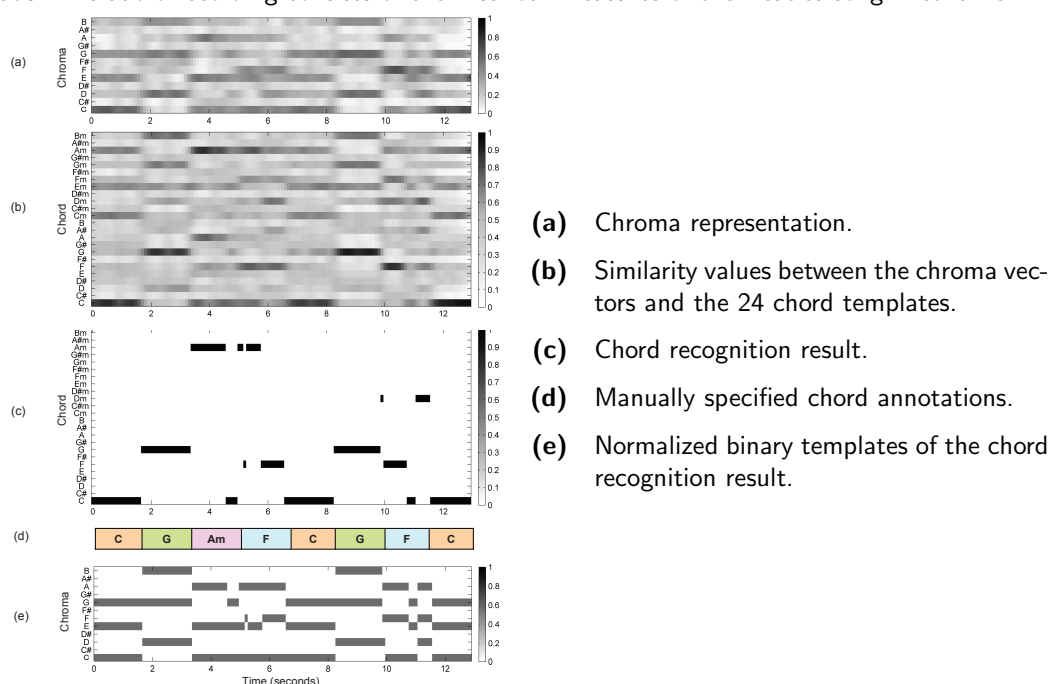
- Note that this measure always yields a value $s(x, y) \in [-1, 1]$. In our case, the vectors x and y only have non-negative entries, thus $s(x, y) \in [0, 1]$.

Lecture PM2L4 Audio

Chapter 6: Generative classification via HMMs – 12 / 55

Template-based chord recognition: an example

We illustrate the above template-based chord recognizer which uses binary templates for the 24 major and minor triads. The audio recording consists of the first four measures of the Beatles song “Let It Be.”



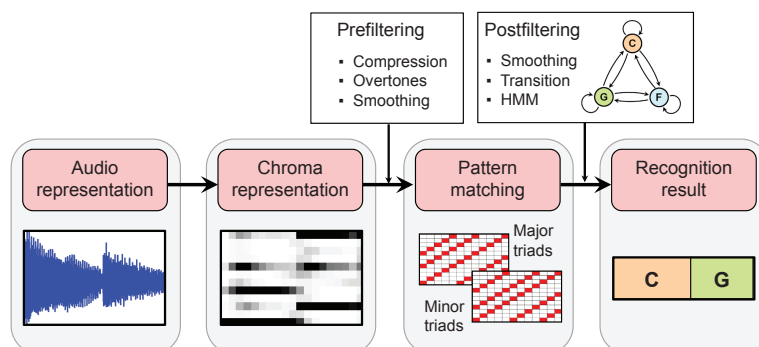
Lecture PM2L4 Audio

Chapter 6: Generative classification via HMMs – 13 / 55

Techniques for improving recognition

To improve the chord recognition results, additional **enhancement techniques** are applied

- either before the pattern matching step (referred to as **prefiltering**)
- or after/within the pattern matching step (referred to as **postfiltering**).



- After sketching some of the enhancements techniques we will discuss in greater detail the hidden Markov model and its impact on improving chord recognition results.

Improved templates with harmonics

- So far, we have used idealized binary chord templates that indicate the presence or absence of notes in the given chord. For real music recordings, however, the presence of **harmonics** and other sound components leads to chroma features where the energy is spread over the chroma bands in a more unstructured, non-binary fashion. This motivates the modification of the chord templates by introducing a model that accounts for harmonics.
- We consider the first eight harmonics and their note correspondences. In the case of a note with chroma C , the first eight harmonics correspond to the chroma values C, C, G, C, E, G, B^b, C .
- Often, the energy in the harmonics decays in an exponential fashion. This can be modeled by assuming that the energy of the k th partial, $k \geq 0$, is α^{k-1} for some $\alpha \in [0, 1]$. From this, one can define a template with harmonics for the chroma C by setting $t_C^h = (1 + \alpha + \alpha^3 + \alpha^7, 0, 0, 0, \alpha^4, 0, 0, \alpha^2 + \alpha^5, 0, 0, \alpha^6, 0)^T$.
- A chord template with harmonics for the major chord **C** is obtained by summing up the templates over the chord's chroma classes:

$$t_C^h = t_C^h + t_E^h + t_G^h.$$

Similarly, one obtains chord templates for the other major and minor chords.

Improved templates from examples

- Instead of explicitly modeling the harmonics, a conceptually different approach is to “learn” chroma patterns for the chord templates from labeled training data. The input of such a learning procedure, which is also known as **supervised learning**, consists of a set of training examples. Each example, in turn, is a pair consisting of an input object and an output value. In our case, the input objects are chroma vectors and the output values are chord labels.
- In the chord recognition context, the simplest way to incorporate knowledge from labeled chroma vectors into the recognition pipeline is to replace the chord templates by averages. To this end, we assume that we have for each $\lambda \in \Lambda$ a set T_λ of chroma vectors labeled with λ . Then, we define the **average template** t_λ^a by taking the average:

$$t_\lambda^a := \frac{1}{|T_\lambda|} \sum_{x \in T_\lambda} x.$$

- The main **advantage** of such a data-driven approach is that the learned chord templates naturally inherit the musical and acoustic properties from the training vectors. In other words, the models automatically adapt to the underlying data.
- On the **downside**, supervised learning requires suitably labeled training data, which should accurately represent the properties of the unseen data to be classified. Such labeled data are often hard to come by.

Feature enhancement strategies

Besides refining and adapting the chord templates, another general strategy is to modify and enhance the chroma features extracted from the audio recordings to be analyzed. In this regard we briefly discuss **spectral and temporal smoothing**.

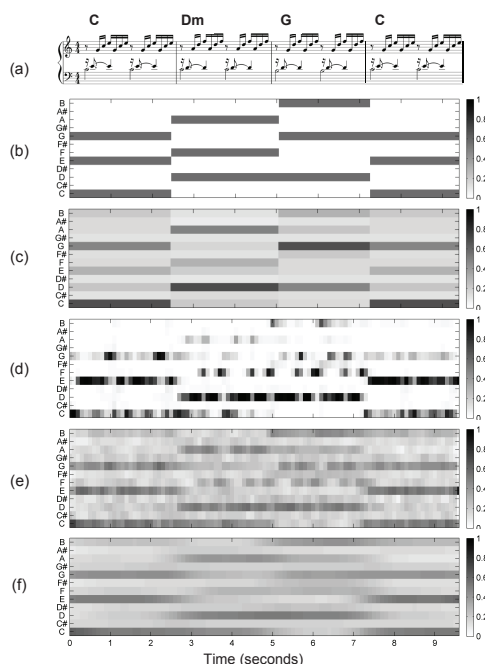
- **Spectral smoothing**: We apply **logarithmic compression** as introduced in Chapter 4. Recall that this strategy makes the signal's chroma distribution more uniform, thus giving smaller components a larger weight relative to the stronger components. Starting with the original chromagram $\mathcal{C}$, the logarithmically compressed chromagram is defined by $\mathcal{C}_\gamma := \Gamma_\gamma \circ \mathcal{C}$, where $\Gamma_\gamma(\nu) := \log(1 + \gamma \cdot \nu)$ for $\nu > 0$. The constant $\gamma > 0$ determines the degree of compression.
- **Temporal smoothing** can be beneficial for reducing the effect of irrelevant local variations. We start with a chromagram given by some sequence $X = (x_1, \dots, x_N)$ consisting of chroma features $x_n \in \mathbb{R}^{12}$. The easiest smoothing strategy is to apply an **averaging filter** (of **length** L) on the sequence. This yields $X^L = (x_1^L, \dots, x_N^L)$, which is defined for odd L and $n \in [1 : N]$ by

$$x_n^L := \frac{1}{L} \sum_{|\ell - n| < L/2} x_\ell.$$

(To avoid boundary problems, use **zero-padding**: $x_n := 0$, for $n \in \mathbb{Z} \setminus [1 : N]$.)

Enhanced chroma features

The following figure shows various types of enhanced chroma features. All chroma vectors are normalized w.r.t. the Euclidean norm and have a 10 Hz resolution.



- (a) Musical score of Prelude BWV 846 in C major by J. S. Bach and reference chord annotations.
- (b) Binary chord templates (corresponding to reference annotations).
- (c) Average chord templates obtained from training data (corresponding to reference annotations).
- (d) Original chroma features obtained from an audio recording.
- (e) Chroma features from (d) after applying logarithmic compression (using $\gamma = 10$).
- (f) Chroma features from (e) after applying smoothing (using $L = 20$).

Lecture PM2L4 Audio

Chapter 6: Generative classification via HMMs – 18 / 55

Evaluation measures

To assess the quality of a chord recognizer, one uses a reference annotation, usually generated by music experts. The resulting frame-wise **reference chord label sequence**

$$(\lambda_1^{\text{Ref}}, \dots, \lambda_N^{\text{Ref}})$$

is then compared with the chord label sequence $(\lambda_1, \dots, \lambda_N)$ computed by the chord recognizer.

- If $\lambda_n = \lambda_n^{\text{Ref}}$, then this label is called **true positive (TP)**, otherwise it is called **false positive (FP)**.
- **Precision P** is the fraction of retrieved chords that are correct.
- **Recall R** is the fraction of correct chords among the retrieved ones.
- The **F-measure** is the harmonic mean of P and R :

$$F := \frac{2 \cdot P \cdot R}{P + R}.$$

Lecture PM2L4 Audio

Chapter 6: Generative classification via HMMs – 19 / 55

Training and validation

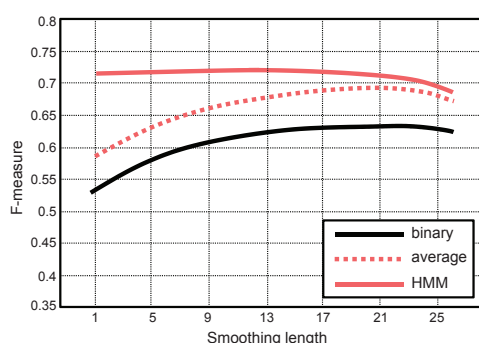
- In practice, training and evaluation are often performed on the basis of the same manually annotated dataset. One typically splits up the available dataset into disjoint subsets called the **training set** and **validation set**.
- In a first stage, the training set is used to learn the chord templates. Then, in the second stage, the validation set is used to evaluate the resulting chord recognition procedure. The goal of such a **cross-validation** approach is to obtain insights into how well the procedure generalizes when applied to unseen data.
- To reduce the dependency of the overall procedure on the partitioning of the dataset, one often performs multiple rounds of cross-validation. One way is to randomly partition the dataset into K subsets of equal size. One of the subsets is used as the validation set, whereas the union of the other $K - 1$ subsets is used as the training set. The cross-validation is repeated K times (the so-called **folds**) with each of the K subsets serving exactly once as the validation set. The K results are then combined, e.g., by taking the average of the considered evaluation measure, to form a single estimate. This approach, which is commonly known as **K -fold cross-validation**, is frequently used to evaluate chord recognition procedures that involve some kind of training.

Lecture PM2L4 Audio

Chapter 6: Generative classification via HMMs – 20 / 55

Effects of temporal smoothing

- The following figure describes the effect of temporal smoothing (based on the fixed chroma type C_{10}) on the chord recognition accuracy.
- The figure shows the average F-measures obtained from a 3-fold cross-validation conducted on the Beatles dataset.
- The results are shown for three different chord recognizers based on binary templates, average templates, and HMMs.



Lecture PM2L4 Audio

Chapter 6: Generative classification via HMMs – 21 / 55

Context-aware classification schemes

- So far, our template-based chord recognizer worked in a purely frame-wise fashion: each frame was classified separately without regarding the previous or future frames.
- In music, however, chord progressions are not arbitrary, but follow certain rules. Some harmonic progressions are more probable than others.
- For example, one often encounters the harmonic progression I - IV - V - I, where tension is built up by traveling from the **tonic** via the **subdominant** to the **dominant**, which resolves again to the tonic. Starting with the C major chord **C** (tonic), it is more likely to find **F** (subdominant) or **G** (dominant) rather than come across harmonically unrelated chords such as **Fm** or **D^b**.
- Furthermore, one often finds the tendency to stay within the same harmonic context for a certain duration. In other words, chord changes are relatively rare events given the high temporal resolution (10 Hz) typically used in the feature extraction step.
- We now study a more refined strategy that performs the filtering in a context-sensitive fashion. The main idea is to introduce a transition model that expresses the likelihood of passing over from one chord to another. This leads us to **Markov chains** and the **hidden Markov model** (HMM).

Lecture PM2L4 Audio

Chapter 6: Generative classification via HMMs – 22 / 55

Markov chains: a motivating example

- Certain transitions from one chord to another are more likely than others.
- **Example:** in a Symphony in **C**, chord progressions like **C** → **F** or **G** → **C** are more likely than **C** → **F[#]**. Markov chains are a helpful (but simplistic) concept to capture such likelihoods.
- Starting point is a finite set $\mathcal{A}$ of chords (generally called **states**). E.g., $\mathcal{A}$ could consist of all 12 **major scales** and all 12 **minor scales**:

C **C[#]** **D** **D[#]** **E** **F** **F[#]** **G** **G[#]** **A** **A[#]** **B**
Cm **C[#]m** **Dm** **D[#]m** **Em** **Fm** **F[#]m** **Gm** **G[#]m** **Am** **A[#]m** **Bm** .

- A progression of chords is then a sequence of states: $(s_1, s_2, s_3, \dots)$ with $s_i \in \mathcal{A}$.
- The change from one state to another is specified according to a set of probabilities associated with each state. In general, a probabilistic description of such a system can be quite complex. To simplify, the Markov model makes the assumption that the probability of a change from the current state s_n to the next state s_{n+1} only depends on the current state, and not on the events that preceded it.

Lecture PM2L4 Audio

Chapter 6: Generative classification via HMMs – 23 / 55

Markov chains: the model

A (finite) **Markov chain** is a triple $(\mathcal{A}, A, C)$ consisting of

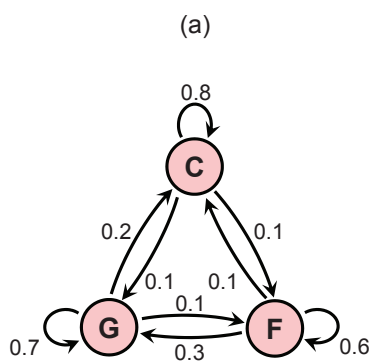
- a finite set $\mathcal{A}$ of **states**: $\mathcal{A} = \{\alpha_1, \dots, \alpha_I\}$,
- a matrix $A = (a_{ij}) \in [0, 1]^{I \times I}$ of **state transition probabilities**, where a_{ij} denotes the probability of switching in one step from state α_i to state α_j , and
- a vector $C = (c_1, \dots, c_I)$ of **initial state probabilities**, i.e., c_i is the probability that a sequence of states $(s_1, s_2, s_3, \dots)$ starts with $s_1 = \alpha_i$.
- Both A and C obey the **standard stochastic constraint**:

$$\sum_{j=1}^I a_{ij} = 1 \quad (\text{for all } i \in [1 : I]) \quad \text{and} \quad \sum_{j=1}^I c_j = 1.$$

Theorem 4. Given the above Markov chain and a fixed N , the probability of observing the state sequence $(\alpha_{i_1}, \alpha_{i_2}, \dots, \alpha_{i_N})$ equals the product of the edge weights:

$$\bullet \xrightarrow{c_{i_1}} \alpha_{i_1} \xrightarrow{a_{i_1, i_2}} \alpha_{i_2} \xrightarrow{a_{i_2, i_3}} \alpha_{i_3} \xrightarrow{a_{i_3, i_4}} \dots \xrightarrow{a_{i_{N-2}, i_{N-1}}} \alpha_{i_{N-1}} \xrightarrow{a_{i_{N-1}, i_N}} \alpha_{i_N}.$$

Illustration of a Markov chain



(b)

	$\alpha_1 = \text{C}$	$\alpha_2 = \text{G}$	$\alpha_3 = \text{F}$
$\alpha_1 = \text{C}$	$a_{11} = 0.8$	$a_{12} = 0.1$	$a_{13} = 0.1$
$\alpha_2 = \text{G}$	$a_{21} = 0.2$	$a_{22} = 0.7$	$a_{23} = 0.1$
$\alpha_3 = \text{F}$	$a_{31} = 0.1$	$a_{32} = 0.3$	$a_{33} = 0.6$

- Illustration of a Markov chain with three states (but without initial state probabilities).
- (a) Graph representation of the Markov chain.
- (b) State transition probability matrix.

Markov chains: an example

- We consider a simple 3-state Markov model of the weather.
- We assume that once a day (e.g., at noon), the observed state of the weather is one of the following:

State 1: rain (or snow) **State 2:** cloudy **State 3:** sunny.

- We assume that the matrix of state transition probabilities is

$$A = (a_{ij}) = \begin{bmatrix} 0.4 & 0.3 & 0.3 \\ 0.2 & 0.6 & 0.2 \\ 0.1 & 0.1 & 0.8 \end{bmatrix}.$$

- Given (!) that the weather on day 1 is sunny (state 3), what is the probability that the weather for the next 7 days will be **sun-sun-rain-rain-sun-cloudy-sun**. More formally, we are asking for the probability of the state sequence $(\alpha_3, \alpha_3, \alpha_3, \alpha_1, \alpha_1, \alpha_3, \alpha_2, \alpha_3)$. As we already know that α_3 is the first state, the relevant weighted path is

$$\alpha_3 \xrightarrow{a_{33}} \alpha_3 \xrightarrow{a_{33}} \alpha_3 \xrightarrow{a_{31}} \alpha_1 \xrightarrow{a_{11}} \alpha_1 \xrightarrow{a_{13}} \alpha_3 \xrightarrow{a_{32}} \alpha_2 \xrightarrow{a_{23}} \alpha_3,$$

i.e., the probability is equal to $0.8 \cdot 0.8 \cdot 0.1 \cdot 0.4 \cdot 0.3 \cdot 0.1 \cdot 0.2 = 1.5360 \cdot 10^{-4}$.

Lecture PM2L4 Audio

Chapter 6: Generative classification via HMMs – 26 / 55

The need to extend the Markov model

- Based on a Markov chain, we can compute a probability for a given observation consisting of a sequence of states or chord types.
- In our chord recognition scenario, however, this is not what we need. Rather than observing a sequence of chord types, we observe a sequence of chroma vectors that are somehow related to the chord types. In other words, the state sequence is not directly visible, but only a fuzzier observation sequence that is generated based on the state sequence.
- Furthermore, rather than computing a probability of the observation sequence, the goal is to uncover the relation between the observed feature vectors and the underlying chord types.
- In the following, we extend the concept of Markov chains to a statistical model referred to as a hidden Markov model (HMM). The idea is to represent the relation between the observed feature vectors and the chord types (the states) using a probabilistic framework. Each state is equipped with a probability function that expresses the likelihood for a given chord type to output or emit a certain feature vector. As a result, we obtain a two-layered process consisting of a **hidden layer** and an **observable layer**.

Lecture PM2L4 Audio

Chapter 6: Generative classification via HMMs – 27 / 55

Hidden Markov models

Definition 5. A (discrete) **hidden Markov model (HMM)** is specified by a tuple $\Theta = (\mathcal{A}, A, \mathcal{B}, B, C)$, where

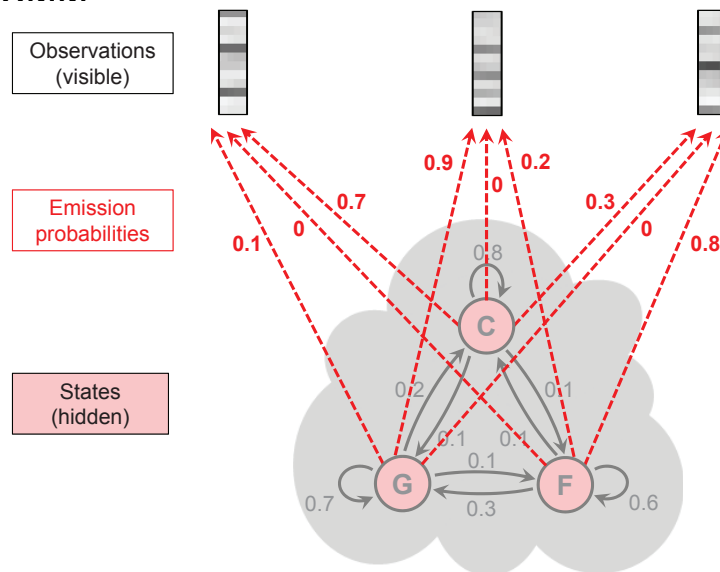
- $\mathcal{A} = \{\alpha_1, \dots, \alpha_I\}$ is a finite set of **states**,
- $A = (a_{ij}) \in [0, 1]^{I \times I}$ is a row stochastic matrix specifying the **state transition probabilities**,
- $\mathcal{B} = \{\beta_1, \dots, \beta_K\}$ is a finite set of **observation symbols**,
- $B = (b_{ik}) \in [0, 1]^{I \times K}$ is a row stochastic matrix specifying the **emission probabilities**,
- $C \in [0, 1]^I$ is the vector of **initial state probabilities**.

The sets $\mathcal{A}$ and $\mathcal{B}$ are usually considered to be fixed components of the model, while (A, B, C) are the free parameters of the model to be determined

- by an expert, or
- by employing a learning procedure based on suitably labeled training data.

As $\mathcal{A}$ and $\mathcal{B}$ are fixed, we often use $\Theta = (A, B, C)$ instead of $\Theta = (\mathcal{A}, A, \mathcal{B}, B, C)$.

Illustration of an HMM



- Illustration of a hidden Markov model extending the example from Slide 25.
- The state-dependent emission probabilities are indicated by the labels of the dashed arrows.

HMM-based generation of observations

We describe an algorithm for generating an observation sequence of length N by an HMM.

Input: HMM specified by $(\mathcal{A}, A, \mathcal{B}, B, C)$

Output: Observation sequence $O = (o_1, \dots, o_N)$

Procedure:

- (1) Choose an initial state $s_1 = \alpha_i$ for some $i \in [1 : I]$ according to the initial state distribution C .
- (2) Set $n = 1$.
- (3) Choose $o_n = \beta_k$ for some $k \in [1 : K]$ according to the emission probability in state s_n . (If $s_n = \alpha_i$ this is specified by the i th row of B .)
- (4) Transit to the new state $s_{n+1} = \alpha_j$ for some $j \in [1 : I]$ according to the state transition probability at state s_n . (If $s_n = \alpha_i$ this is specified by the i th row of A .)
- (5) Set $n = n + 1$ and return to step (3) if $n < N$.
Otherwise terminate the procedure.

HMM: the probabilistic setting

We describe the probabilistic setting behind the generation of observation sequences of length N by an HMM $\Theta = (\mathcal{A}, A, \mathcal{B}, B, C)$.

- The **sample space** is $\Omega_N = \mathcal{A}^N \times \mathcal{B}^N$.
- **Example:** In our chord recognition scenario, $\mathcal{A}^N$ specifies the length N hidden chord sequences, whereas $\mathcal{B}^N$ is the set of all possible length N (quantized) chromagram vector sequences.
- The **probability function** P_Θ^N depends both on the HMM Θ and on the length N and is given by

$$P_\Theta^N(\alpha_{i_1}, \dots, \alpha_{i_N}, \beta_{k_1}, \dots, \beta_{k_N}) = c_{i_1} \cdot b_{i_1, k_1} \prod_{n=2}^N a_{i_{n-1}, i_n} \cdot b_{i_n, k_n}.$$

- In terms of weighted digraphs, this probability value equals the product of the modified edge weights (compare with Theorem 4):

$$\bullet \xrightarrow{c_{i_1} \cdot b_{i_1, k_1}} \alpha_{i_1} \xrightarrow{a_{i_1, i_2} \cdot b_{i_2, k_2}} \alpha_{i_2} \xrightarrow{a_{i_2, i_3} \cdot b_{i_3, k_3}} \alpha_{i_3} \dots \alpha_{i_{N-1}} \xrightarrow{a_{i_{N-1}, i_N} \cdot b_{i_N, k_N}} \alpha_{i_N}.$$

HMM: evaluation and model specification

Prior to applying HMMs we have to discuss three fundamental problems:

- **Evaluation problem:** Given an HMM $\Theta = (\mathcal{A}, A, \mathcal{B}, B, C)$ and an observation sequence $O = (o_1, \dots, o_N)$. The task is to compute the probability of observing O , i.e., we have to compute

$$P_{\Theta}^N(O) := P_{\Theta}^N(\mathcal{A}^N \times \{O\}) = \sum_{S \in \mathcal{A}^N} P_{\Theta}^N(S, O).$$

- ◆ Efficient solution: the **forward procedure**, see below.

- **Uncovering problem:** Given an HMM Θ and an observation sequence $O = (o_1, \dots, o_N)$. Find the single state sequence $S = (s_1, \dots, s_N)$ that “best explains” the observation sequence, i.e.,

$$S = \operatorname{argmax}_{S' \in \mathcal{A}^N} P_{\Theta}^N(S', O).$$

- ◆ Efficient solution: the **Viterbi algorithm**, see below.

- **Estimation problem:** Given an observation sequence $O = (o_1, \dots, o_N)$, the objective is to determine the three probability measures A , B , and C that maximize the probability $P_{\Theta}^N(O)$.

- ◆ We sketch the **Baum-Welch algorithm**, which at least yields a locally optimal solution of the estimation problem.

HMM: the evaluation problem

Let $\mathcal{A} = \{\alpha_1, \dots, \alpha_I\}$, $\mathcal{B} = \{\beta_1, \dots, \beta_K\}$, and $\Theta = (\mathcal{A}, A, \mathcal{B}, B, C)$ an HMM.

- For an observation sequence $O = (o_1, \dots, o_N) = (\beta_{k_1}, \dots, \beta_{k_N})$ we obtain

$$P_{\Theta}^N(O) := P_{\Theta}^N(\mathcal{A}^N \times \{O\}) = \sum_{(i_1, \dots, i_N) \in [1:I]^N} P_{\Theta}^N(\alpha_{i_1}, \dots, \alpha_{i_N}, \beta_{k_1}, \dots, \beta_{k_N}).$$

- A direct evaluation of $P_{\Theta}^N(O)$ along this formula is computationally infeasible even for a small N , since there are I^N summands.
- A more efficient evaluation procedure is based on the following.

Shrinkage Lemma. $P_{\Theta}^1(\alpha_{i_1}, \beta_{k_1}) = c_{i_1} \cdot b_{i_1, k_1}$ and if $N \geq 2$ then

$$P_{\Theta}^N(\alpha_{i_1}, \dots, \alpha_{i_N}, \beta_{k_1}, \dots, \beta_{k_N}) = P_{\Theta}^{N-1}(\alpha_{i_1}, \dots, \alpha_{i_{N-1}}, \beta_{k_1}, \dots, \beta_{k_{N-1}}) \cdot a_{i_{N-1}, i_N} \cdot b_{i_N, k_N}.$$

Proof. Follows from the definition. □

Efficient evaluation: the forward procedure

- For $n \in [1 : N]$ let $O_n := (o_1, \dots, o_n) = (\beta_{k_1}, \dots, \beta_{k_n})$ denote the partial observation sequence up to time n . For $n \in [1 : N]$ and $i \in [1 : I]$ we consider

$$f_n(i) := P_{\Theta}^n(\mathcal{A}^{n-1} \times \{\alpha_i\} \times \{O_n\}).$$

This is the probability of observing the partial observation sequence O_n up to time n and the state α_i at time n . Then

$$P_{\Theta}^N(O) := P_{\Theta}^N(\mathcal{A}^N \times \{O_N\}) = \sum_{i=1}^I P_{\Theta}^N(\mathcal{A}^{N-1} \times \{\alpha_i\} \times \{O_N\}) = \sum_{i=1}^I f_N(i)$$

indicates an alternative way to compute $P_{\Theta}^N(O)$.

- **Initialization:** $f_1(i) = P_{\Theta}^1(\alpha_i, \beta_{k_1}) = c_i \cdot b_{i,k_1}$, for all $i \in [1 : I]$. (This follows from the definition of P_{Θ}^1 .)
- **Induction:** $f_{n+1}(j) = (\sum_{i=1}^I f_n(i) \cdot a_{ij}) \cdot b_{j,k_{n+1}}$, for $n \in [1 : N - 1]$ and $j \in [1 : I]$.

Proof of the induction formula

For $n \in [1 : N - 1]$ and $j \in [1 : I]$ we obtain with the Shrinkage Lemma (SL)

$$\begin{aligned} f_{n+1}(j) &= P_{\Theta}^{n+1}(\mathcal{A}^n \times \{\alpha_j\} \times \{O_{n+1}\}) \\ &= \sum_{i \leq I} P_{\Theta}^{n+1}(\mathcal{A}^{n-1} \times \{\alpha_i\} \times \{\alpha_j\} \times \{O_{n+1}\}) \\ &\stackrel{\text{SL}}{=} \sum_{i \leq I} P_{\Theta}^n(\mathcal{A}^{n-1} \times \{\alpha_i\} \times \{O_n\}) \cdot a_{ij} \cdot b_{j,k_{n+1}} \\ &= \left(\sum_{i \leq I} f_n(i) \cdot a_{ij} \right) \cdot b_{j,k_{n+1}}. \quad \square \end{aligned}$$

Computational complexity of the forward procedure:

- **Start:** Computing $f_1(i)$ for all $i \in [1 : I]$ costs I multiplications.
- **Induction:** Computing $f_n(i)$ for all $i \in [1 : I]$ and all $n \in [2 : N]$ costs $(N - 1) \cdot I \cdot (I + 1)$ multiplications and $(N - 1) \cdot I \cdot (I - 1)$ additions.
- **Termination:** Computing $P_{\Theta}^N(O) = \sum_{i=1}^I f_N(i)$ costs $I - 1$ additions.
- Thus the forward procedure requires a number of operations on the order of $I^2 N$ (instead of I^N)!

Forward procedure: trellis diagram

- The efficient solution of the evaluation problem for HMMs via the forward procedure is illustrated for the parameters $I = 3$, $N = 3$, and the observation sequence $O = (\beta_{k_1}, \beta_{k_2}, \beta_{k_3})$.
- In the next figure, the **weight of a path** is the product of all its edge weights and the **value of a node** v is the sum of all the weights of all paths from the leftmost node to the node v .

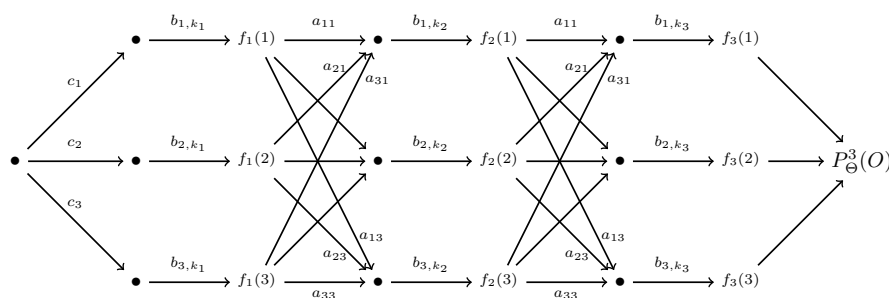


Figure 1: Trellis diagram illustrating the forward procedure.

HMM: the uncovering problem

Let $\mathcal{A} = \{\alpha_1, \dots, \alpha_I\}$, $\mathcal{B} = \{\beta_1, \dots, \beta_K\}$, and $\Theta = (\mathcal{A}, A, \mathcal{B}, B, C)$ an HMM.

- For an observation sequence $O = (o_1, \dots, o_N) = (\beta_{k_1}, \dots, \beta_{k_N})$ we are looking for the single state sequence $S^* = (s_1^*, \dots, s_N^*)$ that “best explains” the observation sequence, i.e.,

$$S^* = \operatorname{argmax}_{S \in \mathcal{A}^N} P_{\Theta}^N(S, O).$$

- The naive approach, which computes $P_{\Theta}^N(S, O)$, for all $S \in \mathcal{A}^N$, and looks for a probability-maximizing S^* , is computationally infeasible even for a small N , since there are I^N state sequences of length N .
- Fortunately, there is a technique, known as the **Viterbi algorithm**, for finding the optimizing state sequence in a much more efficient way.
- The Viterbi algorithm, which is similar to the DTW algorithm, is based on **dynamic programming**. The idea is to recursively compute an optimal (i.e., probability-maximizing) state sequence from optimal solutions for subproblems, where one considers truncated versions of the observation sequence.

Viterbi algorithm: preliminaries (1)

We adhere to the above notation. In particular, $O_n = (o_1, \dots, o_n) = (\beta_{k_1}, \dots, \beta_{k_n})$, for $n \in [1 : N]$ is a truncated version of $O = O_N$.

- Let $P^* := \max_{S \in \mathcal{A}^N} P_{\Theta}^N(S, O)$. With the help of a matrix $\mathbf{D} \in [0, 1]^{I \times N}$ we compute P^* and then S^* . For $i \in [1 : I]$ and $n \in [1 : N]$ we define

$$\mathbf{D}(i, n) := \max_{S \in \mathcal{A}^{n-1}} P_{\Theta}^n(S, \alpha_i, O_n).$$

Thus $\mathbf{D}(i, n)$ is the highest probability along a single state sequence $(s_1, \dots, s_n)$ that accounts for the first n observations and ends in state $s_n = \alpha_i$. The state sequence yielding the maximal value

$$P^* = \max_{i \in [1 : I]} \mathbf{D}(i, N) \quad (1)$$

is the solution to our uncovering problem.

- Our next goal is to show that the matrix $\mathbf{D}$ can be computed recursively along the column index $n \in [1 : N]$.

Viterbi algorithm: preliminaries (2)

Theorem 6. For the matrix $\mathbf{D}$ the following holds.

- (a) $\mathbf{D}(i, 1) = c_i \cdot b_{i, k_1}$, for $i \in [1 : I]$.
- (b) $\mathbf{D}(i, n) = b_{i, k_n} \cdot \max_{j \leq I} a_{ji} \cdot \mathbf{D}(j, n-1)$, for $i \in [1 : I]$ and $n \in [2 : N]$.

Proof. (a): by definition of $\mathbf{D}$ and P_{Θ}^1 we obtain

$$\mathbf{D}(i, 1) = \max_{S \in \mathcal{A}^{1-1}} P_{\Theta}^1(S, \alpha_i, O_1) = P_{\Theta}^1(\alpha_i, \beta_{k_1}) = c_i \cdot b_{i, k_1}.$$

(b) With the Shrinkage Lemma (SL) we obtain

$$\begin{aligned} \mathbf{D}(i, n) &:= \max_{S \in \mathcal{A}^{n-1}} P_{\Theta}^n(S, \alpha_i, O_n) \\ &= \max_{j \leq I} \max_{S' \in \mathcal{A}^{n-2}} P_{\Theta}^n(S', \alpha_j, \alpha_i, O_n) \\ &\stackrel{\text{SL}}{=} \max_{j \leq I} \max_{S' \in \mathcal{A}^{n-2}} P_{\Theta}^{n-1}(S', \alpha_j, O_{n-1}) \cdot a_{ji} \cdot b_{i, k_n} \\ &= b_{i, k_n} \cdot \max_{j \leq I} a_{ji} \cdot \mathbf{D}(j, n-1). \quad \square \end{aligned}$$

Viterbi algorithm: preliminaries (3)

- From the matrix $\mathbf{D}$, we obtain the maximal probability $P^* = \max_{i \leq I} \mathbf{D}(i, N)$.
- However, we do not yet know the optimal state sequence. Again, as with finding the optimal warping path in the case of DTW, we need to apply a **backtracking procedure** that constructs the optimal state sequence in reverse order.
- Let $S^* = (\alpha_{i_1}, \dots, \alpha_{i_N})$ denote the optimal state sequence to be constructed. Then, by the above formula for P^* , the last element α_{i_N} is determined by

$$i_N := \operatorname{argmax}_{j \leq I} \mathbf{D}(j, N).$$

- Furthermore, the element α_{i_n} for $n = N - 1, N - 2, \dots, 1$ is determined by the maximizing argument in the recursion (b) in Theorem 6:

$$i_n = \operatorname{argmax}_{j \leq I} (a_{j, i_{n+1}} \cdot \mathbf{D}(j, n)).$$

- As indicated on the next slide, the backtracking can be simplified by introducing an $I \times (N - 1)$ matrix $\mathbf{E}$ that keeps track of the maximizing argument in the recursion (b) in Theorem 6.

The Viterbi algorithm

Input: HMM Θ specified by $(\mathcal{A}, A, \mathcal{B}, B, C)$

Observation sequence $O = (o_1 = \beta_{k_1}, \dots, o_N = \beta_{k_N})$

Output: Optimal state sequence $S^* = (s_1^*, \dots, s_N^*)$

Procedure:

- (1) Initialize the $I \times N$ matrix $\mathbf{D}$ by $\mathbf{D}(i, 1) = c_i \cdot b_{i, k_1}$ for $i \in [1 : I]$.
- (2) Recursively compute in a nested loop for $n = 2, \dots, N$ and $i = 1, \dots, I$:

$$\mathbf{D}(i, n) = b_{i, k_n} \cdot \max_{j \leq I} a_{ji} \cdot \mathbf{D}(j, n - 1)$$

$$\mathbf{E}(i, n - 1) = \operatorname{argmax}_{j \leq I} a_{ji} \cdot \mathbf{D}(j, n - 1).$$
- (3) Set $i_N = \operatorname{argmax}_{j \leq I} \mathbf{D}(j, N)$.
- (4) Compute for decreasing $n = N - 1, \dots, 1$ the maximizing indices

$$i_n = \mathbf{E}(i_{n+1}, n).$$
- (5) Set $S^* = (s_1^*, \dots, s_N^*) = (\alpha_{i_1}, \dots, \alpha_{i_N})$.

Trellis diagram for computing P^*

The computation of P^* as part of an efficient solution of the uncovering problem for HMMs via the Viterbi algorithm is illustrated for the parameters $I = 3$, $N = 3$, and the observation sequence $O = (\beta_{k_1}, \beta_{k_2}, \beta_{k_3})$.

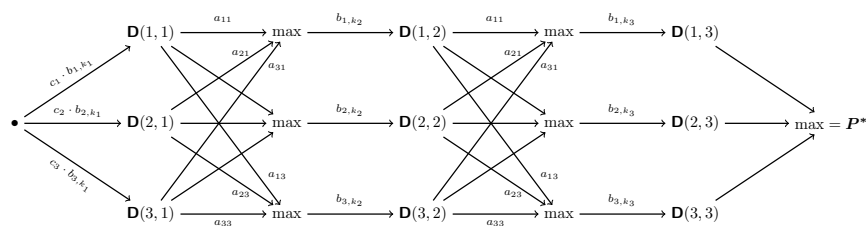


Figure 2: Trellis diagram illustrating the computation of P^* via the Viterbi algorithm.

Trellis diagram for computing S^*

Replacing in the last figure every max by the corresponding argmax (shown below in red and green), the optimal state sequence S^* can be constructed via backtracking along the edges in reverse direction. In our example, we obtain $S^* = (\alpha_3, \alpha_1, \alpha_2)$.

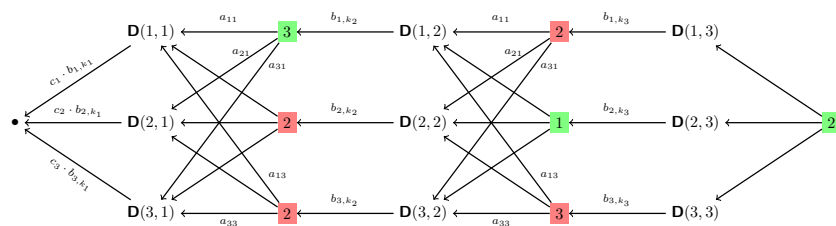
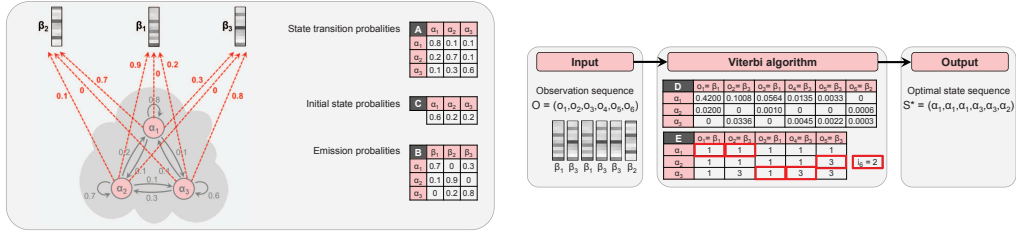


Figure 3: Trellis diagram illustrating the computation of S^* via the Viterbi algorithm.

Illustration of the Viterbi algorithm



■ Viterbi: $O = (\beta_1, \beta_3, \beta_1, \beta_3, \beta_3, \beta_2) \mapsto S^* = (\alpha_1, \alpha_1, \alpha_1, \alpha_3, \alpha_3, \alpha_2)$.

- This example illustrates the kind of **context-sensitive smoothing** introduced by the HMM. Without regarding the previous and subsequent observations, the second observation $o_2 = \beta_3$ is best explained by the state α_3 (with an emission probability of $b_{33} = 0.8$). However, being preceded by $o_1 = \beta_1$ as well as succeeded by $o_3 = \beta_1$, the HMM-based procedure favors the state α_1 to explain $o_2 = \beta_3$ (even though the emission probability is only $b_{13} = 0.3$). In other words, it is beneficial to stay in state α_1 for the first three time instances rather than to switch back and forth between state α_1 and α_3 . The HMM trades off the loss in emission probability against the gain in transition probability, thus reducing the number of transitions between different states.

The estimation problem: preliminaries

- Given an observation sequence $O \in \mathcal{B}^N$, the objective is to determine the three free model parameter, i.e., the probability measures specified by A , B , and C of $\Theta = (\mathcal{A}, A, \mathcal{B}, B, C)$ that maximize the probability

$$P_{\Theta}^N(O) = P_{\Theta}^N(\mathcal{A}^N \times \{O\}).$$

As both $\mathcal{A}$ and $\mathcal{B}$ are fixed, we will write $P_{A,B,C}^N$ instead of P_{Θ}^N .

- To find (A, B, C) that maximizes $P_{A,B,C}^N(O)$ is a typical instance of an optimization problem, where the observation sequence serves as “training material” for adjusting the HMM.
- There is no known way to explicitly solve the given optimization problem. However, iterative procedures that find **locally optimal solutions** have been suggested. One of these procedures is known as the **Baum-Welch algorithm**.
- We will only sketch the iteration step of the Baum-Welch algorithm. This step maps (A, B, C) to another triple (A', B', C') satisfying

$$P_{A',B',C'}^N(O) \geq P_{A,B,C}^N(O).$$

The Baum-Welch algorithm (1)

Let $\Theta = (\mathcal{A}, A, \mathcal{B}, B, C)$ be an HMM and $O = (\beta_{k_1}, \dots, \beta_{k_N}) \in \mathcal{B}^N$ the observation sequence.

- We need the following subsets of $\mathcal{A}^N$ (here $i \in [1 : I]$ and $n \in [1 : N]$ resp. $n \in [1 : N - 1]$):

$$\mathcal{A}_n^N(i) := \{S \in \mathcal{A}^N \mid s_n = \alpha_i\} \quad \text{and} \quad \mathcal{A}_n^N(i, j) := \{S \in \mathcal{A}^N \mid s_n = \alpha_i, s_{n+1} = \alpha_j\}.$$

- For $i \in [1 : I]$ and $n \in [1 : N]$ resp. $n \in [1 : N - 1]$ define the conditional probabilities

$$\gamma_n(i) := P_{A,B,C}^N(\mathcal{A}_n^N(i) \times \{O\} \mid \mathcal{A}^N \times \{O\})$$

$$\Gamma_n(i, j) := P_{A,B,C}^N(\mathcal{A}_n^N(i, j) \times \{O\} \mid \mathcal{A}^N \times \{O\}).$$

Thus, given O (i.e., the event $\mathcal{A}^N \times \{O\}$ has occurred), $\gamma_n(i)$ (resp. $\Gamma_n(i, j)$) is the probability of being in state α_i at a given time instance n (and in state α_j at time $n + 1$).

- As $\mathcal{A}^N = \bigsqcup_{i \leq I} \mathcal{A}_n^N(i) = \bigsqcup_{i, j \leq I} \mathcal{A}_n^N(i, j)$, we deduce that $\sum_{i \leq I} \gamma_n(i) = 1$, i.e., γ_n is a probability measure on $[1 : I]$, for $n \in [1 : N]$. Similarly, Γ_n is a probability measure on $[1 : I]^2$, for each $n \in [1 : N - 1]$.

The Baum-Welch algorithm (2)

Now we can specify the update rules of the Baum-Welch algorithm:

- The vector $C' = (c'_1, \dots, c'_I)$ of **updated initial state probabilities** is given by

$$c'_i = \gamma_1(i).$$

- The matrix $A' = (a'_{ij}) \in [0, 1]^{I \times I}$ of **updated state transition probabilities** is given by

$$a'_{ij} = \frac{\sum_{n < N} \Gamma_n(i, j)}{\sum_{n < N} \gamma_n(i)}.$$

- The matrix $B' = (b'_{ik}) \in [0, 1]^{I \times K}$ of **updated emission probabilities** is given by

$$b'_{ik} = \frac{\sum_{n \leq N} \gamma_n(i) \cdot \delta(o_n, \beta_k)}{\sum_{n \leq N} \gamma_n(i)}.$$

Here $\delta(o_n, \beta_k) := 1$, if $o_n = \beta_k$, and $\delta(o_n, \beta_k) = 0$, otherwise. (Kronecker- δ)

- It is left as an exercise to prove that A' , B' , and C' are row stochastic.
- The triple (A', B', C') is called the **Baum-Welch update** of (A, B, C) .

Maximum-likelihood estimation of HMM parameters

Theorem 7 (Baum & Eagon). Let (A', B', C') denote the Baum-Welch update of (A, B, C) .

- (a) If $(A', B', C') \neq (A, B, C)$, then $P_{A', B', C'}^N(O) > P_{A, B, C}^N(O)$.
- (b) Iterating the update process of the Baum-Welch algorithm, the resulting triples $(A^{(n)}, B^{(n)}, C^{(n)})$ converge to a triple $(\hat{A}, \hat{B}, \hat{C})$.
- (c) The triple $(\hat{A}, \hat{B}, \hat{C})$ yields a local maximum of the likelihood-function $(A, B, C) \mapsto P_{A, B, C}^N(O)$.

Proof. See Baum & Sell: Growth Transformations for Functions on Manifolds, Pacific Journal of Mathematics, Vol. 27, No. 2, 1968, pp. 211–227. $\square$

- The final result of this reestimation procedure is called a **maximum-likelihood** estimate of the parameters of the HMM.
- In practice, this local maximum may be far from the global maximum.
- Furthermore, the final estimate delivered by the Baum-Welch algorithm crucially depends on the initialization of the HMM to start with.

Lecture PM2L4 Audio

Chapter 6: Generative classification via HMMs – 48 / 55

HMM-based chord recognition

We now show how the concept of HMMs can be applied to our chord recognition scenario. First of all, we need to create a (discrete) HMM $\Theta = (\mathcal{A}, A, \mathcal{B}, B, C)$ that suitably models our chord recognition problem.

- The **states** of the HMM are used to model the various chord types that are allowed in the recognition problem. In the following, we consider the twelve major and minor triads, thus setting

$$\mathcal{A} = (\alpha_1, \dots, \alpha_I) := \{\mathbf{C}, \mathbf{C}^\sharp, \mathbf{D}, \dots, \mathbf{B}, \mathbf{Cm}, \mathbf{C}^\sharp\mathbf{m}, \mathbf{Dm}, \dots, \mathbf{Bm}\} \quad (2)$$

In this case, the HMM consists of $I = 24$ states, which we enumerate as indicated by (2). For example, α_1 corresponds to $\mathbf{C}$ and α_{13} to $\mathbf{Cm}$.

- A **transition probability matrix** $A = (a_{ij}) \in [0, 1]^{I \times I}$, where a_{ij} expresses the probability for the transition from state (chord) α_i to state (chord) α_j , can be specified in many ways. For example, the matrix may be defined manually by a music expert based on rules from harmony theory.
- The most common approach is to generate such a matrix automatically by estimating the transition probabilities from labeled data. In the following, we describe such an automated approach.

Lecture PM2L4 Audio

Chapter 6: Generative classification via HMMs – 49 / 55

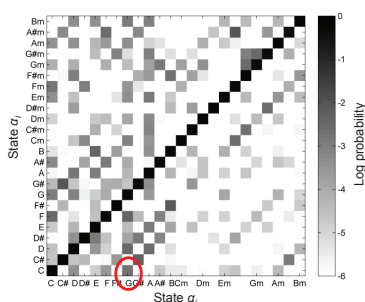
Automated specification of Transition Probabilities

- For an automated approach, one needs suitably labeled training data. Our experiments are based on an annotated Beatles dataset.
- In the following, we assume that each audio recording of the training dataset is represented by a sequence of frames and that each frame is labeled with one of the states (chords) in $\mathcal{A}$. Thus we obtain a sequence of chord labels: $(\alpha_{i_1}, \dots, \alpha_{i_N})$.
- Next, we count how often each of the 24×24 possible chord transitions occurs in the training data. To this end, we consider adjacent elements $(\alpha_{i_n}, \alpha_{i_{n+1}})$, so called **bigrams**. Let μ_{ij} denote the number of bigrams of the form (α_i, α_j) among the $(\alpha_{i_n}, \alpha_{i_{n+1}})$. Finally, we define the transition probability matrix $A = (a_{ij}) \in [0, 1]^{I \times I}$ by

$$a_{ij} := \frac{\mu_{ij}}{\sum_{k \in [1:I]} \mu_{ik}}.$$

- The numerator of this fraction is the total number of bigrams (α_i, α_j) , whereas the denominator counts the total number of bigrams starting at α_i and going to an arbitrary state.
- Note that this definition corresponds to the update rule $A \rightsquigarrow A'$ in the Baum-Welch algorithm.

Example: estimated transition probability matrix

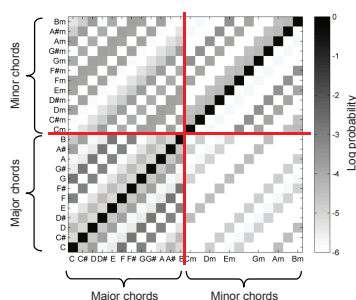


- Estimated transition probability matrix (using a log probability scale for visualization purposes). The matrix was computed using the Beatles dataset.
- As an example, the coefficient $a_{1,7}$ (corresponding to the transition $C \rightarrow G$) has been highlighted.

- Note that the matrix is dominated by the coefficients on the main diagonal, which correspond to self-transitions $\alpha_i \rightarrow \alpha_i$. **Explanation:** Most of the chord durations occurring in the training data are much longer (often around one second) than the frame length in use (100 ms corresponding to a feature rate of 10 Hz).
- The transition probability matrix is more irregular than expected. The reason is that the training set used is not balanced with regard to the various existing musical keys. For example, the Beatles dataset may contain many songs in the **C** major or **A** minor scales, but rarely a song in the **D^b** major or **B^b** minor scales. As a result, there is not sufficient training data for obtaining robust estimates of certain transition probabilities.

Transposition-invariant transition probabilities

- To make the training set more balanced and invariant to the underlying musical key, we apply a **cyclic chroma shift** to the considered bigrams so that the first chord of each bigram corresponds either to **C** or to **Cm**. In this case, we say that the bigrams have been **normalized** with respect to the chroma **C**.
- For example, after normalization, the transitions **C[♯] → G[♯]** or **D → A** become **C → G**. Afterwards, one takes twelve versions of each of the resulting **C**-normalized bigrams. These twelve versions are then normalized with respect to the twelve different chroma values.
- Furthermore, the statistics on the number of transitions have become invariant under transpositions (cyclic chroma shifts). The resulting **transposition-invariant** state transition probability matrix is shown below.



Lecture PM2L4 Audio

Chapter 6: Generative classification via HMMs – 52 / 55

Specification of observation symbols

- Next we have to specify a finite set $\mathcal{B} = \{\beta_1, \dots, \beta_K\}$ of observation symbols. In our chord recognition scenario, the observations are chroma vectors that have previously been extracted from the given audio recording. In other words, the observations are 12-dimensional real-valued vectors which are elements of the continuous feature space $\mathcal{F} = \mathbb{R}^{12}$.
- So far, we have only considered the case of **discrete HMMs**, where the observations are discrete symbols coming from a **finite output space** $\mathcal{B}$. To make discrete HMMs applicable to our scenario, one possible procedure is to introduce a finite set of prototype vectors, a so-called **codebook**. Such a codebook can be regarded as a discretization of the continuous feature space $\mathcal{F} = \mathbb{R}^{12}$, where each prototype vector represents an entire range of feature vectors. The process of mapping an arbitrary feature vector to one of the prototype vectors is also referred to as **quantization**.
- There are many ways for determining a codebook and for performing the quantization. Many approaches are based on **clustering** techniques, where a set of given vectors is grouped in such a way that vectors in the same group (called a cluster) are more similar to each other than to those in other clusters. The prototype vectors of the codebook are then defined as, e.g., the centroids of the resulting clusters. Furthermore, in the quantization, an arbitrary vector is mapped to the most similar (or closest) prototype vector.

Lecture PM2L4 Audio

Chapter 6: Generative classification via HMMs – 53 / 55

Continuous HMMs

- Finding a suitable codebook as well as a suitable quantization function is in general not an easy problem, and there are many design choices to be made. Furthermore, the quantization may introduce serious degradations, in particular for vectors that are not represented well by the codebook.
- An alternative is to use **continuous HMMs**, where the finite output space $\mathcal{B}$ is replaced by a continuous output space. In this case, the emission probabilities specified by the matrix B are to be replaced by **continuous probability density functions** (PDFs).
- Also the reestimation procedure needs to be modified by considering values of the density functions instead of discrete probabilities. The PDFs used in practice are typically based on **Gaussian functions** or mixtures thereof. Such PDFs can be compactly described by a few parameters (means, variances, mixture coefficients). Similar to the Baum-Welch algorithm, one can derive an iterative estimation procedure that adjusts the PDF parameters (instead of the emission probabilities as in the case of discrete HMMs) on the basis of given training sequences.

Lecture PM2L4 Audio

Chapter 6: Generative classification via HMMs – 54 / 55

References

To a large extent this chapter is based on Chapter 5 of the very recommendable book:

- **Meinard Müller: Fundamentals of Music Processing, Springer 2015.**

We strongly recommend for deeper studies of HMMs:

- **Rabiner:** A tutorial on hidden Markov models and selected applications in speech recognition, Proceedings of the IEEE, **77** (1989), pp. 257–286.
 - **Huang, Ariki, & Jack:** Hidden Markov Models for Speech Recognition, Edinburgh University Press, 1990.
- For the theoretical foundation of the Baum-Welch algorithm we refer to
- **Baum & Eagon:** An inequality with applications to the statistical estimation for probabilistic functions of Markov processes and to a model for ecology, Bull. Amer. Math. Soc., **70** (1967), pp. 360–363.
 - **Baum & Sell:** Growth Transformations for Functions on Manifolds, Pacific Journal of Mathematics, **27**, No. 2, 1968, pp. 211–227.

Lecture PM2L4 Audio

Chapter 6: Generative classification via HMMs – 55 / 55